

[54] Analyzing Query Optimizer Performance in the Presence and Absence of Cardinality Estimates

요약

본 논문은 Datta 등이 2023년 arXiv에 발표한 논문으로 (arXiv:2311.17293), 카디널리티 추정이 쿼리 옵티마이저의 성능에 미치는 영향을 체계적으로 분석한다. 논문의 핵심은 카디널리티 추정 오류가 실행 계획 선택에 어떻게 영향을 미치는지, 그리고 정확한 카디널리티 정보를 제공했을 때 성능이 얼마나 개선되는지를 정량적으로 평가하는 것이다.

이 연구는 실증적(empirical) 성격의 논문이다. 학습 기반 방법이나 새로운 알고리즘을 제시하기보다는, 기존 데이터베이스 시스템(특히 PostgreSQL)에서 카디널리티 추정이 성능에 미치는 영향을 깊이 있게 분석한다. 이를 통해 다음과 같은 중요한 질문에 답한다:

1. 카디널리티 추정 오류의 심각성은?
2. 카디널리티 추정 오류가 성능 저하를 유발하는 메커니즘은?
3. 정확한 카디널리티 정보로 인한 성능 개선 가능성은?

이러한 분석은 Exqutor와 같은 카디널리티 추정 연구의 실제 가치를 입증한다. 즉, 카디널리티 추정을 개선하는 것이 단순히 추정 오류를 줄이는 것을 넘어서, 실제 쿼리 성능에 어떤 영향을 미치는지 보여준다.

상세분석

54.1 카디널리티 추정 오류의 영향 메커니즘

카디널리티 추정 오류가 쿼리 성능에 영향을 미치는 경로는 다음과 같다:

첫째, 조인 순서 선택이다. 조인 쿼리에서 여러 테이블을 조인할 때, 조인 순서에 따라 중간 결과의 크기가 달라진다. 예를 들어: - A JOIN B JOIN C에서 - (A JOIN B) JOIN C 순서와 (B JOIN A) JOIN C 순서에서의 중간 결과 크기가 다름

옵티마이저는 카디널리티 추정을 기반으로 조인 순서를 결정한다. 만약 카디널리티 추정이 부정확하면, 최악의 조인 순서를 선택할 수 있다. 특히 여러 테이블의 조인에서는 추정 오류가 누적되어 매우 나쁜 순서를 선택할 수 있

다.

둘째, 조인 방법 선택이다. 각 조인에 대해 여러 조인 알고리즘(Nested Loop Join, Hash Join, Merge Join 등)을 사용할 수 있다. 각 알고리즘의 효율성은 입력 크기에 따라 달라진다: - Nested Loop Join: 외부 입력이 작을 때 효율적 - Hash Join: 해시 테이블이 메모리에 들어갈 때 효율적 - Merge Join: 입력이 정렬되어 있을 때 효율적

잘못된 카디널리티 추정은 부적절한 조인 알고리즘 선택을 유발한다. 예를 들어, 입력이 실제로 크지만 크다고 추정되지 않으면, 비효율적인 Nested Loop Join을 선택할 수 있다.

셋째, 물리 연산자(physical operator)의 선택이다. 스캔 방법(풀 테이블 스캔 vs. 인덱스 스캔), 정렬 방법(메모리 정렬 vs. 디스크 정렬 등)의 선택이 카디널리티에 따라 달라진다.

54.2 실험 설계와 데이터셋

논문의 실험은 체계적으로 설계되었다. PostgreSQL을 대상 데이터베이스로 사용하고, 여러 벤치마크 워크로드를 사용한다:

첫째, Join Order Benchmark (JOB)이다. 이는 IMDB 영화 데이터베이스를 기반으로 한 벤치마크로, 복잡한 조인 쿼리들을 포함한다. JOB의 쿼리들은 매우 도전적인 것으로 알려져 있으며, 옵티마이저들이 종종 나쁜 계획을 선택한다.

둘째, TPC-H 및 TPC-DS 벤치마크이다. 이들은 비즈니스 쿼리 처리를 시뮬레이션하는 표준 벤치마크이다.

셋째, 합성 데이터(synthetic data)이다. 논문은 다양한 데이터 분포(균등, 스쿠드, 클러스터링 등)를 가진 합성 데이터도 생성하여 사용한다.

54.3 카디널리티 추정 오류의 정량화

논문은 추정 오류를 여러 방식으로 정량화한다:

첫째, q-error이다. 이미 언급했듯이, q-error는 예측값과 실제값의 최대 비율을 나타낸다. 논문의 실험에서 PostgreSQL의 카디널리티 추정 오류는: - 중간값(median): 1.5배 - 평균값: 약 10배 - 95 백분위수: 수십 배에서 수백 배

둘째, 오류의 방향(direction)이다. 추정값이 실제값보다 큰 과다 추정(overestimation)인지, 아니면 과소 추정(underestimation)인지는 다른 영향을 미친다: - 과다 추정: Nested Loop Join 선택, 비효율적인 연산자 선택 - 과소 추정: 인덱스 사용을 피함, 메모리 부족 상황 유발

셋째, 오류의 누적이다. 여러 단계의 조인에서 각 단계의 추정 오류가 누적되어, 전체 오류가 매우 심각해질 수 있다.

54.4 성능 영향 분석

논문의 핵심 결과는 카디널리티 추정 오류가 실제 쿼리 성능에 미치는 영향을 정량화한 것이다:

첫째, 극단적 성능 저하이다. 일부 쿼리에서는 부정확한 카디널리티 추정으로 인해 실행 시간이 1000배 이상 증가한다. 이는 몇 밀리초에 완료되어야 할 쿼리가 초 단위로 실행된다는 뜻이다.

둘째, 시스템 수준의 영향이다. 한두 개의 느린 쿼리는 전체 시스템의 처리량(throughput)에 미치는 영향이 크다. 멀티테넌트 환경이나 OLTP 시스템에서는 이러한 느린 쿼리가 다른 쿼리들을 블로킹할 수 있다.

셋째, 성능 개선의 가능성이다. 정확한 카디널리티 정보를 옵티마이저에게 제공하면: - 평균 성능: 5배 개선 - 최악의 경우: 100배 이상 개선 가능

54.5 카디널리티 추정 오류의 원인

논문은 PostgreSQL의 카디널리티 추정이 부정확한 이유를 분석한다:

첫째, 통계의 부정확성이다. PostgreSQL은 열(column)별로 히스토그램을 유지하지만: - 히스토그램의 버킷 수가 제한적 (보통 100) - 극값(extreme values)이 제대로 표현되지 않음 - 열 간의 상관관계가 고려되지 않음

둘째, 독립성 가정이다. 여러 조건이 결합될 때, 옵티마이저는 각 조건이 독립적이라고 가정하고 확률을 곱한다. 실제로는 많은 조건들이 상관관계가 있다.

셋째, 복잡한 쿼리의 어려움이다. 많은 조인이 포함된 복잡한 쿼리에서는 각 단계의 추정 오류가 누적된다.

54.6 본 논문과의 관계

이 논문은 Exqutor의 동기와 중요성을 입증하는 역할을 한다. 구체적으로:

첫째, 카디널리티 추정이 중요함을 보여준다. 카디널리티 추정은 단순히 통계적 흥미로운 문제가 아니라, 실제 데이터베이스 성능에 극적인 영향을 미치는 중요한 문제다.

둘째, 기존 방법의 한계를 명확히 한다. 전통적인 통계 기반 추정의 평균 10배 오류는 시스템 수준에서 용인 불가능한 수준이다.

셋째, 새로운 접근 방식의 필요성을 강조한다. 머신러닝 기반의 카디널리티 추정 (예: SelNet, Exqutor)이 이러한 문제를 해결할 수 있음을 시사한다.

Exqutor의 맥락에서 보면, 이 논문의 분석은 유사성 쿼리의 카디널리티 추정이 얼마나 중요한지를 강조한다. 벡터 데이터베이스 시스템도 유사한 문제를 가지고 있을 것이고, Exqutor의 머신러닝 기반 접근이 이를 해결할 수 있음을 시사한다.

추가 제기 문제

1. **적응형 쿼리 실행:** 실행 중에 카디널리티 추정을 수정하고 계획을 변경할 수 있는 적응형(adaptive) 쿼리 처리의 이점은?

2. **통계 정확도의 유지:** 데이터가 지속적으로 변하는 환경에서, 통계를 정확하게 유지하는 방법은?
3. **학습 기반 방법의 효과:** 이 논문의 발견이 머신러닝 기반 카디널리티 추정의 필요성을 얼마나 강력하게 지지하는가?
4. **종합적 해결책:** 카디널리티 추정만으로 충분한가, 아니면 비용 모델(cost model)의 개선도 동시에 필요한가?